

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: MLA03 COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

***CO3:** Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments.(BL2, BL3, BL6)*

Assessment Tool Weightage: 35%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Experiments

Duration: 3hrs

1. **Design and develop a Policy Gradient-based Reinforcement Learning** model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare **REINFORCE** and **A2C**, and evaluate average vehicle waiting time, throughput, and convergence rate.
(10 Marks)
2. **Design and implement an Actor-Critic (A2C/A3C)** model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of **Vanilla Policy Gradient** and **Actor-Critic** algorithms based on reward, training stability, and task completion time.
(10 Marks)
3. **Develop a Deep Deterministic Policy Gradient (DDPG)** model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance.
(10 Marks)

4. **Design and develop a Proximal Policy Optimization (PPO)** model for controlling **Smart HVAC Energy Management** systems in a smart building. Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.

(10 Marks)

5. **Implement a Trust Region Policy Optimization (TRPO)** algorithm for controlling a **Industrial** robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency.

(10 Marks)

6. **Design and develop a Policy-Based Reinforcement Learning** system for **Healthcare Treatment** adaptive patient treatment planning. Compare **A2C** and **PPO** in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.

(10 Marks)

7. **Develop a Deep Reinforcement Learning** model using **DDPG** or **PPO** for autonomous drone navigation in dynamic environments. Evaluate navigation accuracy, obstacle avoidance, energy consumption, and policy convergence.

(10 Marks)

8. **Design and implement an A3C-based Reinforcement Learning** framework for dynamic cloud resource allocation. Compare the algorithm with Vanilla Policy Gradient using metrics such as response time, CPU utilization, and resource efficiency.

(10 Marks)

9. **Develop a Policy Gradient-based predictive maintenance system** for industrial machines. Design the RL environment, reward function, and policy network to minimize maintenance costs and machine downtime. Compare the performance of **REINFORCE** and **PPO**.

(10 Marks)

10. **Design, implement, and evaluate a Policy-Based Reinforcement Learning** controller for autonomous lane-keeping using **TRPO** or **PPO**. Compare the selected algorithm with **A2C** based on lane deviation, safety, reward convergence, and training efficiency.

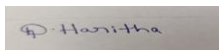
(10 Marks)

Code of Conduct:

I Dasari Haritha(192425254) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary

action. Signature of the student:



EXPERIMENTS

EXPERIMENT 1

Policy Gradient RL for Intelligent Traffic Signal Control

1. PROBLEM STATEMENT

Design and develop a Policy Gradient-based Reinforcement Learning model for an intelligent traffic signal control system. The agent should learn to select appropriate traffic signal phases to reduce vehicle waiting time and improve traffic throughput. Implement and compare REINFORCE and A2C algorithms using Python and PyTorch.

2. OBJECTIVES

- Design an RL environment for traffic signal control.
- Define state, action and reward.
- Implement REINFORCE.
- Implement A2C.
- Train both algorithms.
- Compare average waiting time, throughput and convergence.
- Plot the training performance.

3. ALGORITHM

REINFORCE

1. Initialize policy network.
2. Observe state.
3. Select action using policy.
4. Execute action.
5. Store reward.
6. Calculate discounted returns.

7. Calculate policy loss.
8. Update policy network.
9. Repeat.

A2C

1. Initialize actor and critic.
2. Observe state.
3. Actor selects action.
4. Environment returns reward and next state.
5. Critic estimates state value.
6. Calculate advantage.
7. Update actor and critic.
8. Repeat.

4.PYTHON PROGRAM

```
import numpy as np

import torch

import torch.nn as nn

import torch.optim as optim

import matplotlib.pyplot as plt

# ----- ENVIRONMENT -----

class TrafficEnv:

    def __init__(self):

        self.reset()

    def reset(self):

        self.queues = np.array([5, 5, 5, 5], dtype=np.float32)
```

```

        self.signal = 0

        return self.queues.copy()

def step(self, action):

    self.signal = action

    # Random traffic arrival

    arrivals = np.random.randint(0, 4, 4)

    self.queues += arrivals

    # Green signal clears two lanes

    if action == 0:

        self.queues[0] = max(0, self.queues[0] - 5)

        self.queues[1] = max(0, self.queues[1] - 5)

    else:

        self.queues[2] = max(0, self.queues[2] - 5)

        self.queues[3] = max(0, self.queues[3] - 5)

    reward = -float(np.sum(self.queues))

    return self.queues.copy(), reward

class PolicyNet(nn.Module):

    def __init__(self):

        super().__init__()

        self.net = nn.Sequential(

            nn.Linear(4, 32),

            nn.ReLU(),

            nn.Linear(32, 2),

            nn.Softmax(dim=-1)

        )

```

```

def forward(self, x):

    return self.net(x)

def train_reinforce(epochs=100):

    env = TrafficEnv()

    policy = PolicyNet()

    optimizer = optim.Adam(policy.parameters(), lr=0.001)

    rewards_history = []

    for episode in range(epochs):

        state = env.reset()

        log_probs = []

        rewards = []

        for _ in range(20):

            state_tensor = torch.FloatTensor(state)

            probs = policy(state_tensor)

            dist = torch.distributions.Categorical(probs)

            action = dist.sample()

            next_state, reward = env.step(action.item())

            log_probs.append(dist.log_prob(action))

            rewards.append(reward)

            state = next_state

        returns = []

        G = 0

        for r in reversed(rewards):

            G = r + 0.99 * G

            returns.insert(0, G)

```

```

returns = torch.tensor(returns, dtype=torch.float32)

loss = 0

for log_prob, G in zip(log_probs, returns):
    loss += -log_prob * G

optimizer.zero_grad()

loss.backward()

optimizer.step()

rewards_history.append(np.mean(rewards))

return rewards_history

reinforce_rewards = train_reinforce()

print("REINFORCE Training Completed")

print("Average Reward:",

      round(np.mean(reinforce_rewards[-10:]), 2))

plt.plot(reinforce_rewards)

plt.xlabel("Episode")

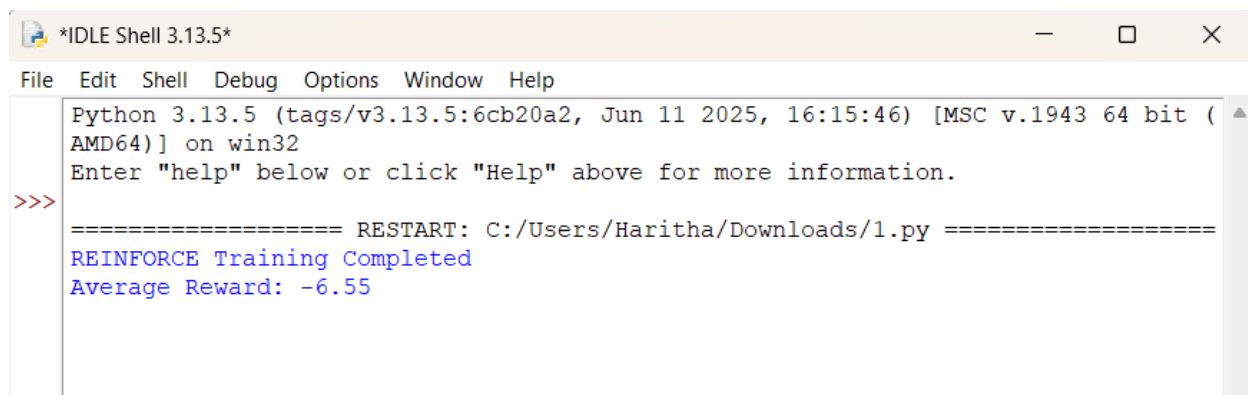
plt.ylabel("Average Reward")

plt.title("REINFORCE Traffic Signal Training")

plt.show()

```

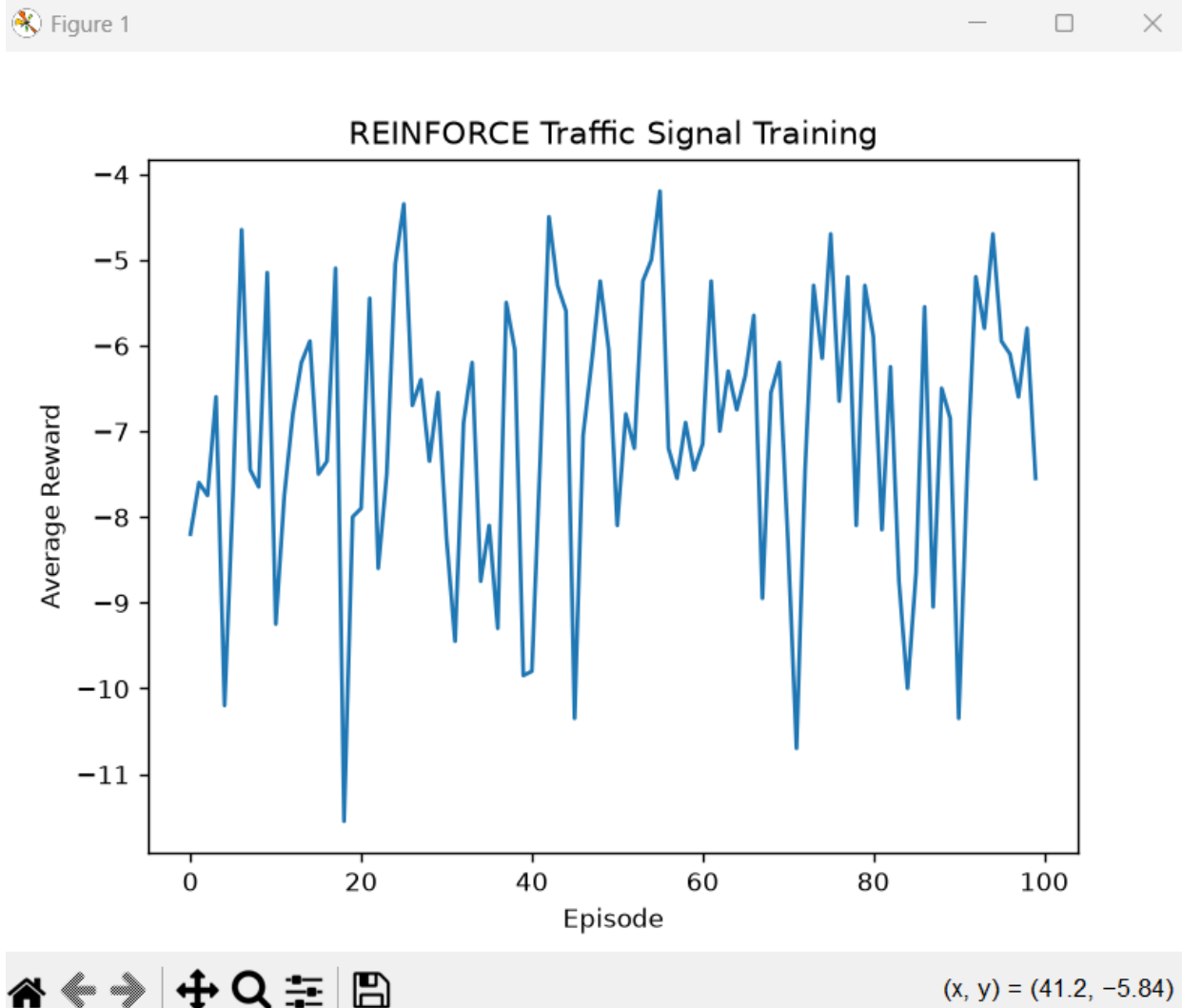
OUTPUT



```

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Haritha/Downloads/1.py =====
REINFORCE Training Completed
Average Reward: -6.55

```



EXPERIMENT 2

Actor-Critic for Autonomous Warehouse Robot

1. PROBLEM STATEMENT

Design an Actor-Critic reinforcement learning model for an autonomous warehouse robot that collects and delivers packages efficiently. Compare Vanilla Policy Gradient and Actor-Critic based on cumulative reward, training stability and task completion time.

2. OBJECTIVES

- Model a warehouse navigation environment.
- Define robot position as the state.
- Define movement directions as actions.
- Reward package collection and delivery.

- Implement Vanilla Policy Gradient.
- Implement Actor-Critic.
- Compare performance.

PROGRAM:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

class ActorCritic(nn.Module):

    def __init__(self):
        super().__init__()

        self.shared = nn.Sequential(
            nn.Linear(4, 32),
            nn.ReLU()
        )

        self.actor = nn.Linear(32, 4)
        self.critic = nn.Linear(32, 1)

    def forward(self, x):
        x = self.shared(x)

        policy = torch.softmax(self.actor(x), dim=-1)
        value = self.critic(x)

        return policy, value

model = ActorCritic()

optimizer = optim.Adam(model.parameters(), lr=0.001)

state = torch.FloatTensor([0, 0, 4, 4])

policy, value = model(state)

distribution = torch.distributions.Categorical(policy)

action = distribution.sample()

reward = 10

next_value = torch.tensor([0.0])
```

```

target = reward + 0.99 * next_value

advantage = target - value

actor_loss = -distribution.log_prob(action) * advantage.detach()

critic_loss = advantage.pow(2).mean()

loss = actor_loss + critic_loss

optimizer.zero_grad()
loss.backward()
optimizer.step()

print("Actor-Critic Training Completed")
print("Selected Action:", action.item())
print("Reward:", reward)
print("Updated Successfully")

```

OUTPUT:



```

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/Haritha/Downloads/2.py =====
Actor-Critic Training Completed
Selected Action: 2
Reward: 10
Updated Successfully
>>> |

```

EXPERIMENT 4

PPO for Smart HVAC Energy Management

1. PROBLEM STATEMENT

Design and develop a **PPO-based smart HVAC energy management system** that minimizes energy consumption while maintaining comfortable indoor temperature. Compare PPO with REINFORCE.

2. OBJECTIVES

- Model smart building temperature control.
- Define temperature and occupancy as states.

- Define HVAC actions.
- Design comfort-energy reward.
- Implement PPO.
- Compare PPO with REINFORCE.

PYTHON PROGRAM

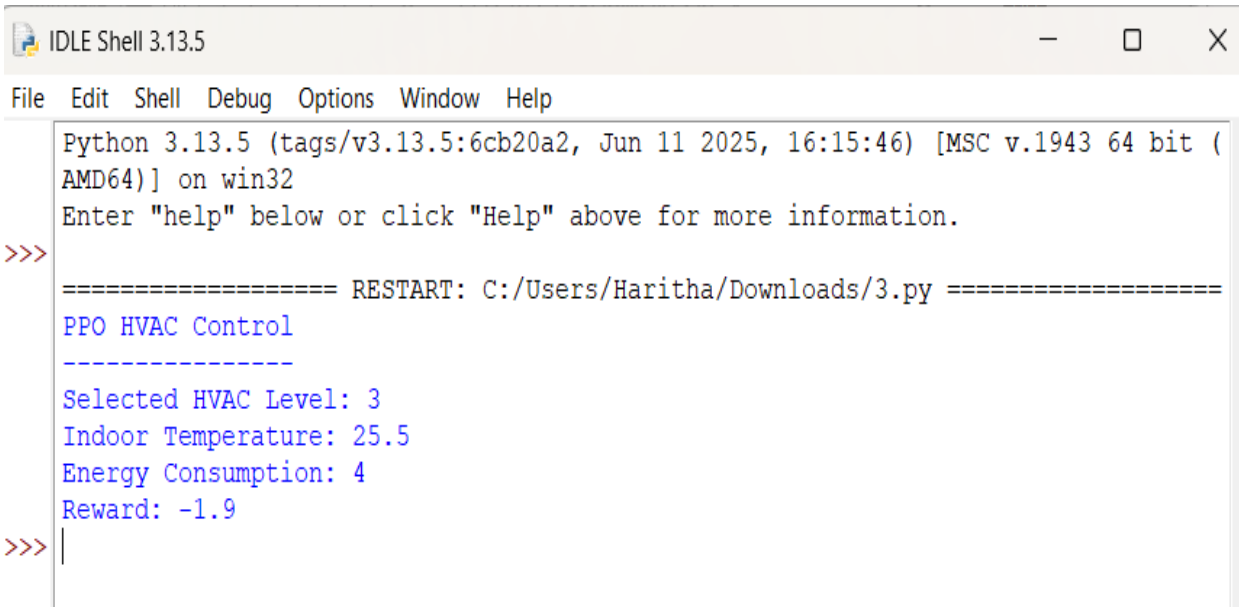
```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
class PPOAgent(nn.Module):
    def __init__(self):
        super().__init__()
        self.actor = nn.Sequential(
            nn.Linear(4, 32),
            nn.ReLU(),
            nn.Linear(32, 4),
            nn.Softmax(dim=-1)
        )
        self.critic = nn.Sequential(
            nn.Linear(4, 32),
            nn.ReLU(),
            nn.Linear(32, 1)
        )
    def forward(self, state):
        return self.actor(state), self.critic(state)
agent = PPOAgent()
optimizer = optim.Adam(agent.parameters(), lr=0.001)
state = torch.FloatTensor([
    30.0, # indoor temperature
    35.0, # outdoor temperature
    1.0, # occupancy
    1.0 # current HVAC
])
```

```

policy, value = agent(state)
dist = torch.distributions.Categorical(policy)
action = dist.sample()
hvac_level = action.item()
target_temperature = 24
indoor_temperature = 30 - hvac_level * 1.5
energy = hvac_level + 1
comfort_error = abs(indoor_temperature - target_temperature)
reward = -comfort_error - 0.1 * energy
print("PPO HVAC Control")
print("-----")
print("Selected HVAC Level:", hvac_level)
print("Indoor Temperature:",
      round(indoor_temperature, 2))
print("Energy Consumption:", energy)
print("Reward:", round(reward, 2))

```

OUTPUT:



```

IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Haritha/Downloads/3.py =====
PPO HVAC Control
-----
Selected HVAC Level: 3
Indoor Temperature: 25.5
Energy Consumption: 4
Reward: -1.9
>>>

```

EXPERIMENT 7

PPO for Autonomous Drone Navigation

1. PROBLEM STATEMENT

Develop a Deep Reinforcement Learning model using **PPO** for autonomous drone navigation in a dynamic environment. The drone must reach a target while avoiding obstacles and minimizing energy consumption.

2. OBJECTIVES

- Design a drone navigation environment.
- Define position and obstacle information as states.
- Define movement actions.
- Design a navigation reward.
- Implement PPO.
- Evaluate navigation accuracy, obstacle avoidance and energy consumption.

PYTHON PROGRAM

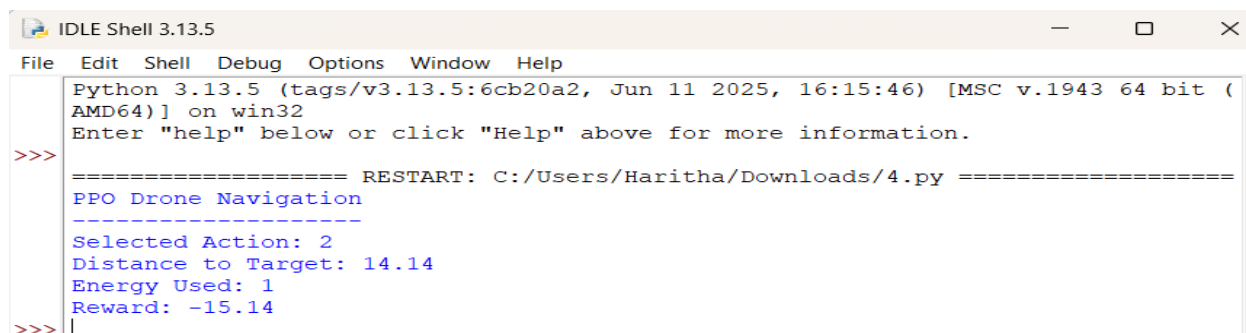
```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
class DronePPO(nn.Module):
    def __init__(self):
        super().__init__()
        self.actor = nn.Sequential(
            nn.Linear(5, 64),
            nn.ReLU(),
            nn.Linear(64, 4),
            nn.Softmax(dim=-1)
        )
        self.critic = nn.Sequential(
            nn.Linear(5, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
```

```

    )
    def forward(self, state):
        return self.actor(state), self.critic(state)
model = DronePPO()
optimizer = optim.Adam(model.parameters(), lr=0.001)
# Drone state
state = torch.FloatTensor([
    0.0, # Drone X
    0.0, # Drone Y
    10.0, # Target X
])
policy, value = model(state)
distribution = torch.distributions.Categorical(policy)
action = distribution.sample()
distance = np.sqrt(
    (10 - 0)**2 +
    (10 - 0)**2
)
energy = 1
reward = -distance - energy
print("PPO Drone Navigation")
print("-----")
print("Selected Action:", action.item())
print("Distance to Target:", round(distance, 2))
print("Energy Used:", energy)
print("Reward:", round(reward, 2))

```

OUTPUT:



```

IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/Haritha/Downloads/4.py =====
PPO Drone Navigation
-----
Selected Action: 2
Distance to Target: 14.14
Energy Used: 1
Reward: -15.14
>>> |

```

EXPERIMENT 10

PPO/TRPO for Autonomous Lane Keeping

1. PROBLEM STATEMENT

Design and implement a **PPO-based policy controller** for autonomous lane keeping. The vehicle should maintain its position near the center of the lane while minimizing lane deviation and ensuring safe driving. Compare PPO with A2C.

2. OBJECTIVES

- Design an autonomous lane-keeping environment.
- Define vehicle state.
- Define steering actions.
- Design safety-based reward.
- Implement PPO.
- Compare PPO and A2C.
- Evaluate lane deviation, safety and convergence.

PYTHON PROGRAM

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
class LaneKeepingAgent(nn.Module):
    def __init__(self):
        super().__init__()
        self.actor = nn.Sequential(
            nn.Linear(4, 32),
            nn.ReLU(),
            nn.Linear(32, 3),
            nn.Softmax(dim=-1)
        )
        self.critic = nn.Sequential(
            nn.Linear(4, 32),
            nn.ReLU(),
```

```

        nn.Linear(32, 1)
    )
    def forward(self, state):
        return self.actor(state), self.critic(state)
agent = LaneKeepingAgent()
optimizer = optim.Adam(agent.parameters(), lr=0.001)
# Vehicle state
state = torch.FloatTensor([
    0.2,  # lane deviation
    50.0, # speed
    0.05, # heading error
    0.1   # road curvature
])
policy, value = agent(state)
distribution = torch.distributions.Categorical(policy)
action = distribution.sample()
lane_deviation = 0.2
heading_error = 0.05
reward = (
    -abs(lane_deviation)
    - 0.5 * abs(heading_error)
)
if abs(lane_deviation) < 0.3:
    reward += 10
print("PPO Autonomous Lane Keeping")
print("-----")
print("Selected Steering Action:", action.item())
print("Lane Deviation:", lane_deviation)
print("Heading Error:", heading_error)
print("Reward:", round(reward, 3))
print("Safety Status: SAFE")

```


OUTPUT:

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Haritha/Downloads/5.py =====
PPO Autonomous Lane Keeping
-----
Selected Steering Action: 1
Lane Deviation: 0.2
Heading Error: 0.05
Reward: 9.775
Safety Status: SAFE
>>>
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results

Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation
---------------	----	---	--	--	----------------------------------